

FIR Filter Implemented with the Self-purging Redundancy Technique

DOCUMENTATION

June 2026

1 Introduction

A FIR filter (Finite Impulse Response filter) is a filter whose output depends only on a finite number of current and past input samples. It is described by:

$$y[n] = \sum_{k=0}^{N-1} h[k] x[n-k]$$

where:

- $x[n]$ is the input signal,
- $y[n]$ is the output signal,
- $h[k]$ are the filter coefficients (taps)

Each input sample is multiplied by a coefficient and the products are summed together. In hardware, FIR filters are typically implemented using a chain of multipliers, adders, and delay registers.

Self-Purging Redundancy (SPR) is a fault-tolerance technique that employs multiple redundant instances of a module and automatically excludes faulty instances from the final decision process. The operation of SPR can be summarized as follows:

1. Replicate the module M times.
2. Apply the same input to all replicas.
3. Compare the outputs of all replicas.
4. Identify modules whose outputs agree with the majority.
5. Exclude modules whose outputs deviate beyond a predefined tolerance threshold.
6. Generate the final output using only the trusted replicas.

Here we apply SPR on the FIR filter at the MAC module level - each "tap" has M redundant MAC modules whose outputs are fed to a voter.

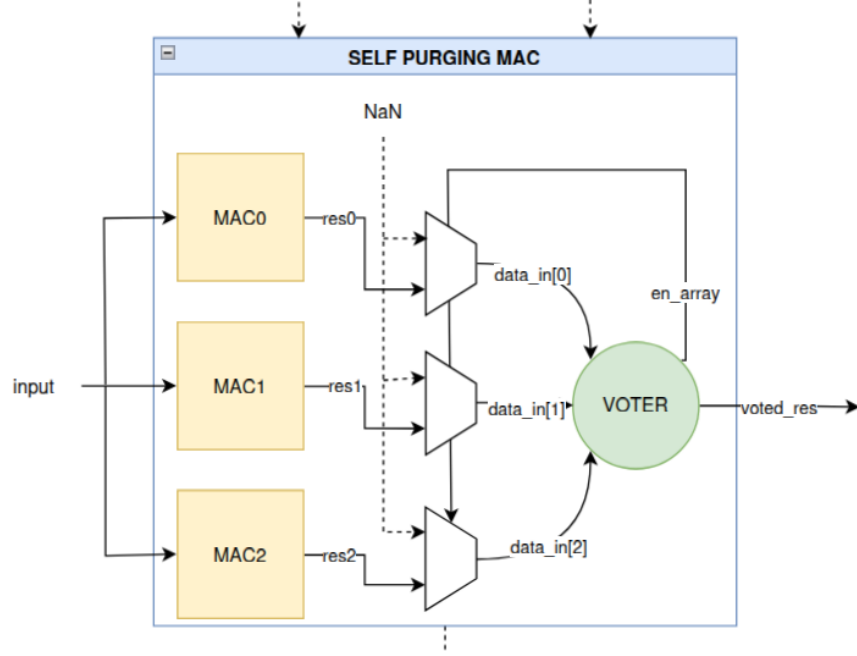


Figure 1: Example of the SPR technique applied to a MAC module

2 Architecture

The designed Self-purging FIR filter was written to be fully parametrized. This means that the input data width, filter order and number of redundant MAC modules can be changed through the constants set in the top module of the project.

Since both the MAC modules and the voter have several pipeline stages, the multiplication inputs for each successive Self-purging MAC module within the filter need to be delayed by the number of pipeline phases present in both the MAC modules and the voter (Figure 2).

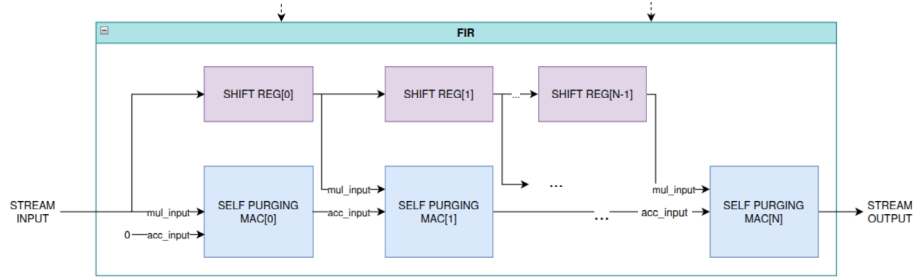


Figure 2: FIR Filter - Modular Design

The implementation of the shift registers is trivial and what we can note is that we use an enable signal to stall the whole filter when the data input is not valid. This signal is visible on Figure 3 for the shift registers as well.

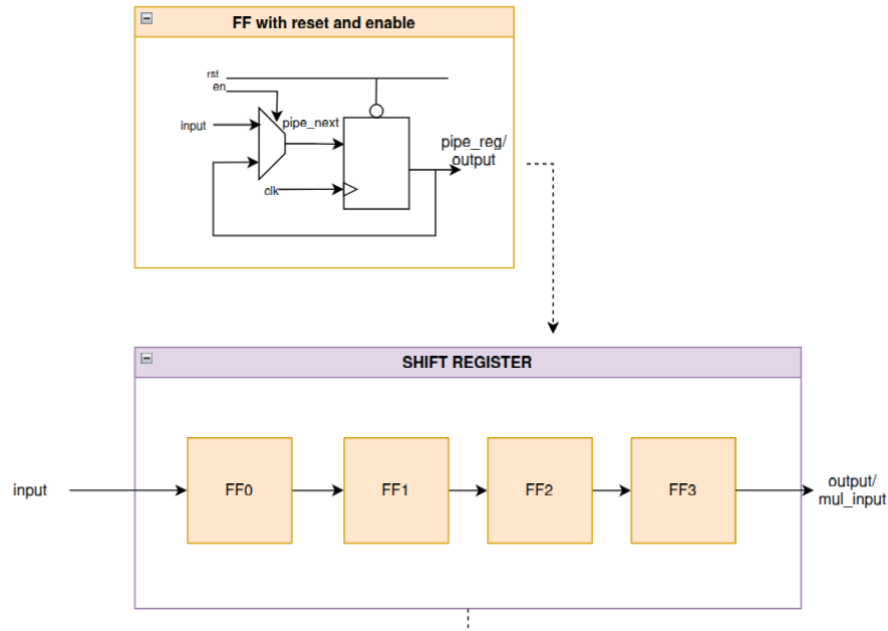


Figure 3: Shift register implementation

The MAC modules are straight forward, a multiply and accumulate operation with a pipeline installed to improve throughput (Figure 4). The outputs are fed to a voter (Figure 5) which compares each input with each other using XOR gates. The tolerance of the comparison can be adjusted by changing the number of bits which are compared. This comparison along with the current values of the inputs determines the switch enable signals and voted output.

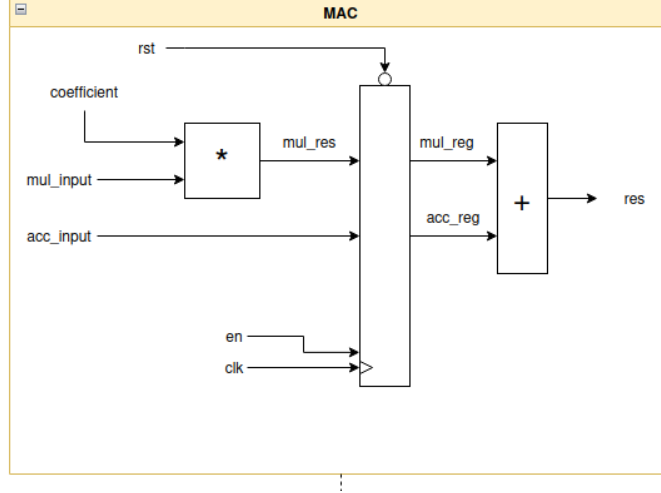


Figure 4: MAC Design

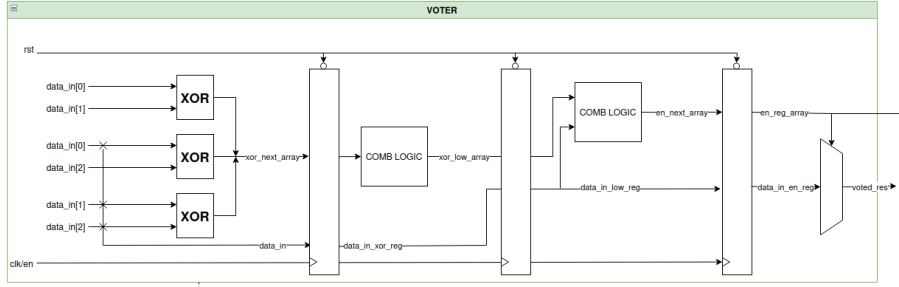


Figure 5: Voter Design

3 Testing results

Running a simulation of the filter for three redundant modules and three "taps" or three coefficients gives us the waveforms shown in Figure 6. The green colored signals are the inputs and outputs of the filter while the yellow, blue, and pink colored signals are, respectively, the inputs and outputs of Self-purging MAC modules. We can clearly see that for each MAC (or for each of the colors) the results are shifted by the number of pipelines we have per MAC module.

If we were to look at the first valid input into the filter we can see that it's corresponding output appears twelve clocks later. This number corresponds to the product of the number of "taps" and the number of pipelines that we have within one self-purging MAC module.

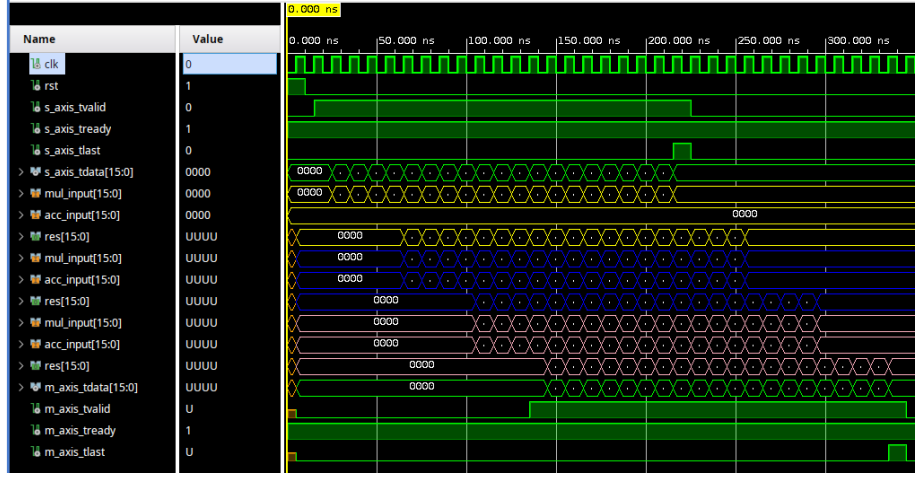


Figure 6: Simulation of filter

To simulate a stuck-at fault in the system we use the tcl command:

```
restart
add_force {'signal_path '} -radix hex {0000}
run all
```

While to undo the effect of the set stuck-at fault previously applied we use the tcl command:

```
remove_force {'signal_path '}
```

We expect to see the same output chosen when there is no fault (Figure 7) as well as when there is a single stuck-at fault (Figure 8). The difference in those two cases would be the switch enable signal, which should disable the MAC with the stuck-at fault. If we had two stuck-at faults and three redundant modules the output voted should be the erroneous one and that will also be reflected in the switch enable signals, which will disable all three MAC modules (Figure 9).

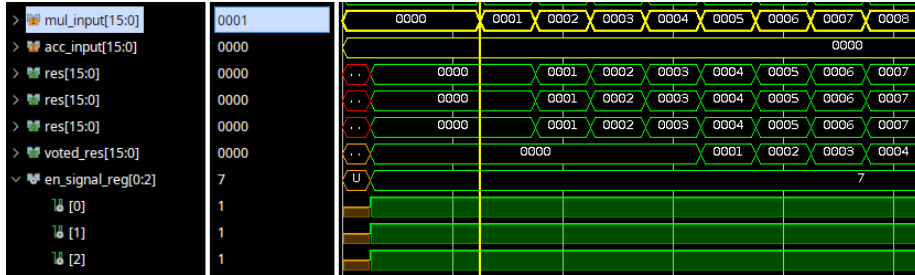


Figure 7: Voted result and switch enable signals when there is no fault present

4 Timing and resource analysis

The inputs to the program include the neural network topology and dimensions, the paths to the training and test datasets, and the number of processes or threads used for execution. In our experiments, we vary the network size and the number of execution threads to examine their effect on training time. The primary objectives are to demonstrate that parallel implementations achieve measurable speedup when the neural network is "sufficiently large", and that this speedup decreases as the network size becomes smaller. We will also show that the change in network size doesn't have a decisive impact on network precision.

The results will be depicted using tables, where each table corresponds to a different network topology listed in the caption using the following notation: **[input : layer 1 dim : ... : layer N dim : output]**

Table 1: Training time and speed-up for neural network [786:512:256:128:10]

Num of threads	Training time [s]	Speed-up	Precision [%]
1	50.071	1.00	4*89.23
2	32.2821	1.55	
4	19.3453	2.59	
8	16.0741	3.11	

Table 2: Training time and speed-up for neural network [786:256:128:64:10]

Num of threads	Training time [s]	Speed-up	Precision [%]
1	16.4427	1.00	4*86.74
2	9.38719	1.75	
4	7.05949	2.33	
8	7.1395	2.30*	

Table 3: Training time and speed-up for neural network [786:256:64:10]

Num of threads	Training time [s]	Speed-up	Precision [%]
1	12.4463	1.00	4*88.26
2	11.6225	1.07	
4	8.14092	1.53	
8	6.13998	2.03	

The speed-up results marked in the table with an asterisk for the 8 thread version of the program are smaller than the results we have for the 4 thread version. The explanation for that could be the hardware limitation. The program is run on an intel i7 10th generation processor which possesses 8 threads in total. Using all of them could be influencing the result as this program isn't

Table 4: Training time and speed-up for neural network [786:128:64:10]

Num of threads	Training time [s]	Speed-up	Precision [%]
1	6.8231	1.00	4*88.31
2	5.41727	1.26	
4	4.56017	1.50	
8	4.88325	1.39*	

the only process running on the processor. This idea could be supported by the results we get when using 6 or 7 threads, making sure to use layer dimensions that are divisible by one of the two numbers as well as by four and eight so that the load is balanced (see table 5 below).

Table 5: Training time and speed-up for neural network [786:120:60:10]

Num of threads	Training time [s]	Speed-up	Precision [%]
1	8.99365	1	4*87.68
2	5.91038	1.52	
4	5.19014	1.73	
6	5.00726	1.80	
8	5.28303	1.70	

Another notable thing is the difference in speed-up when looking at the 4 thread version used for each of the different neural networks. It is getting smaller as the network topology and dimensions get smaller. This is the influence of the overhead we mentioned.

We can thus conclude that the network parallelization successfully sped-up the training process without hindering network functionality.